Software Development

Real-Time Campus Mobility and Ride Management Platform

Participation Format

This is a team problem statement.

Teams may consist of up to the maximum number of 04 participants.

Participants are expected to design and develop a real-time ride management platform that enables passengers and drivers to seamlessly connect within a campus environment.

The challenge evaluates full-stack development, real-time communication systems, database design, API development, state management, and user experience design skills.

---

1. Motivation

Efficient transportation is a critical component of modern campuses, universities, airports, hospitals, logistics hubs, and smart cities.

Users require quick and reliable transportation, while drivers need visibility into ride demand and passenger requests. Without centralized coordination, transportation systems often suffer from inefficient resource utilization, delays, uneven demand distribution, and poor user experiences.

The IIT Roorkee campus spans a large geographical area and relies heavily on e-rickshaws for last-mile transportation. However, ride requests and driver availability are currently coordinated through fragmented and informal mechanisms.

This challenge provides an opportunity to build a real-world mobility platform that mirrors many of the engineering challenges encountered in ride-hailing and dispatch systems, including:

- Real-time communication
- Geospatial data handling
- Ride assignment workflows
- State synchronization

- Dashboard analytics
- Multi-user coordination

Participants will develop a campus-scale ride management platform capable of connecting passengers and drivers through a centralized digital system.

---

2. Problem Statement

Design and develop a responsive web or mobile application that enables passengers and drivers to discover, request, assign, and manage rides within a campus environment.

The platform should provide:

- User authentication
- Driver onboarding
- Ride request management
- Ride assignment workflows
- Real-time ride updates
- Driver analytics and insights

The system should prioritize usability, reliability, and responsiveness while remaining scalable and maintainable.

Participants are encouraged to focus on solving the core ride-management problem effectively rather than attempting to replicate every feature of commercial ride-hailing platforms.

---

3. Functional Requirements

Mandatory Features

A. User Authentication & Profile Management

Implement a secure authentication system.

Passenger Accounts

- Registration
- Login

- Profile Management

Driver Accounts

- Registration
- Login
- Vehicle Information
- Driver Verification Information

Participants may implement:

- JWT Authentication
- Session-Based Authentication
- OAuth Authentication (Optional)

B. Driver Availability Management

Drivers should be able to:

- Go Online
- Go Offline
- Update current availability status

Passengers should be able to view available drivers before requesting rides.

C. Ride Request Workflow

Passengers should be able to:

- Request a ride
- Specify pickup location
- Specify destination
- View ride status

Drivers should be able to:

- View incoming ride requests
- Accept requests
- Reject requests

The system must ensure that a ride can only be assigned to a single driver.

D. Real-Time Ride Updates

Implement real-time communication using technologies such as:

- WebSockets
- Socket.IO
- Server-Sent Events

The platform should support:

- Live ride status updates
- Driver availability updates
- Ride assignment notifications

This is the core engineering component of the challenge.

E. Ride Lifecycle Management

Support complete ride tracking through states such as:

- Requested
- Accepted
- In Progress
- Completed
- Cancelled

The system should maintain a consistent ride state throughout the workflow.

F. Driver Dashboard

Provide a dashboard displaying:

- Total rides completed
- Active rides
- Ride history
- Ratings received
- Basic ride statistics

Visualizations may include:

- Summary Cards
- Charts

- Activity Tables

G. Ratings & Feedback

Passengers should be able to:

- Rate completed rides
- Provide optional written feedback

The platform should maintain:

- Average driver ratings
- Feedback history
- Driver performance summaries

Optional Features

A. Live Map Integration

Display:

- Driver locations
- Pickup points
- Active rides

using mapping solutions such as:

- Leaflet
- OpenStreetMap
- Google Maps

B. Ride Scheduling

Allow users to:

- Schedule rides for future time slots
- Manage upcoming bookings

Examples:

- Early morning classes
- Club rehearsals
- Event transportation

C. Digital Payments

Simulate or integrate:

- UPI Payments
- QR Code Payments
- Payment History

D. Demand Analytics

Analyze historical ride data and identify:

- Peak demand hours
- Popular pickup locations
- Ride demand patterns

E. Demand Forecasting

Use machine learning techniques to predict:

- High-demand periods
- Potential ride hotspots

Examples:

- Linear Regression
- Decision Trees
- Time-Series Models

---

4. Deliverables

Deliverable 1: Working Application

A functional application demonstrating the implemented features.

Deliverable 2: Design Document

Format: PDF

The document should include:

- Problem Understanding
- System Architecture
- Database Schema
- Entity Relationship Diagram (ERD)
- API Overview
- Design Decisions

Maximum Length: 8 Pages

Deliverable 3: Public GitHub Repository

The repository should contain:

- Source Code
- Configuration Files
- Documentation

The solution should be reproducible.

Deliverable 4: README.md

The README should include:

- Project Overview
- Technology Stack
- Setup Instructions
- Running the Application
- Feature List

Deliverable 5: Demonstration Video

Duration: Maximum 3 Minutes

The video should demonstrate:

- User Registration & Login
- Ride Request Workflow
- Ride Assignment Process
- Real-Time Updates
- Driver Dashboard
- Ratings & Feedback System

---

5. Guidelines

1. Prioritize Core Ride Workflows

A reliable ride management system is preferable to numerous incomplete features.

---

2. Focus on Real-Time Communication

The strongest submissions will effectively utilize WebSockets or similar technologies to deliver live updates.

---

3. Maintain Data Consistency

Ride states, assignments, and driver availability should remain synchronized across the platform.

---

4. Design for Scalability

Consider how the system would support larger numbers of users and ride requests.

5. Follow Good Engineering Practices

Strong submissions should demonstrate:

- Clean Code
- Modular Architecture
- Proper Documentation
- Maintainability

---

6. Evaluation Criteria

Criterion	Weightage
Functionality & Feature Completeness	30%
Real-Time System Design	20%
Backend & Database Design	15%
User Experience & Interface Design	15%
Code Quality & Documentation	10%
Innovation & Additional Features	10%

Bonus Considerations

Additional credit may be awarded for:

- Live Map Integration
- Ride Scheduling
- Digital Payments
- Demand Analytics
- Demand Forecasting
- Advanced Dashboard Features

Type your text
